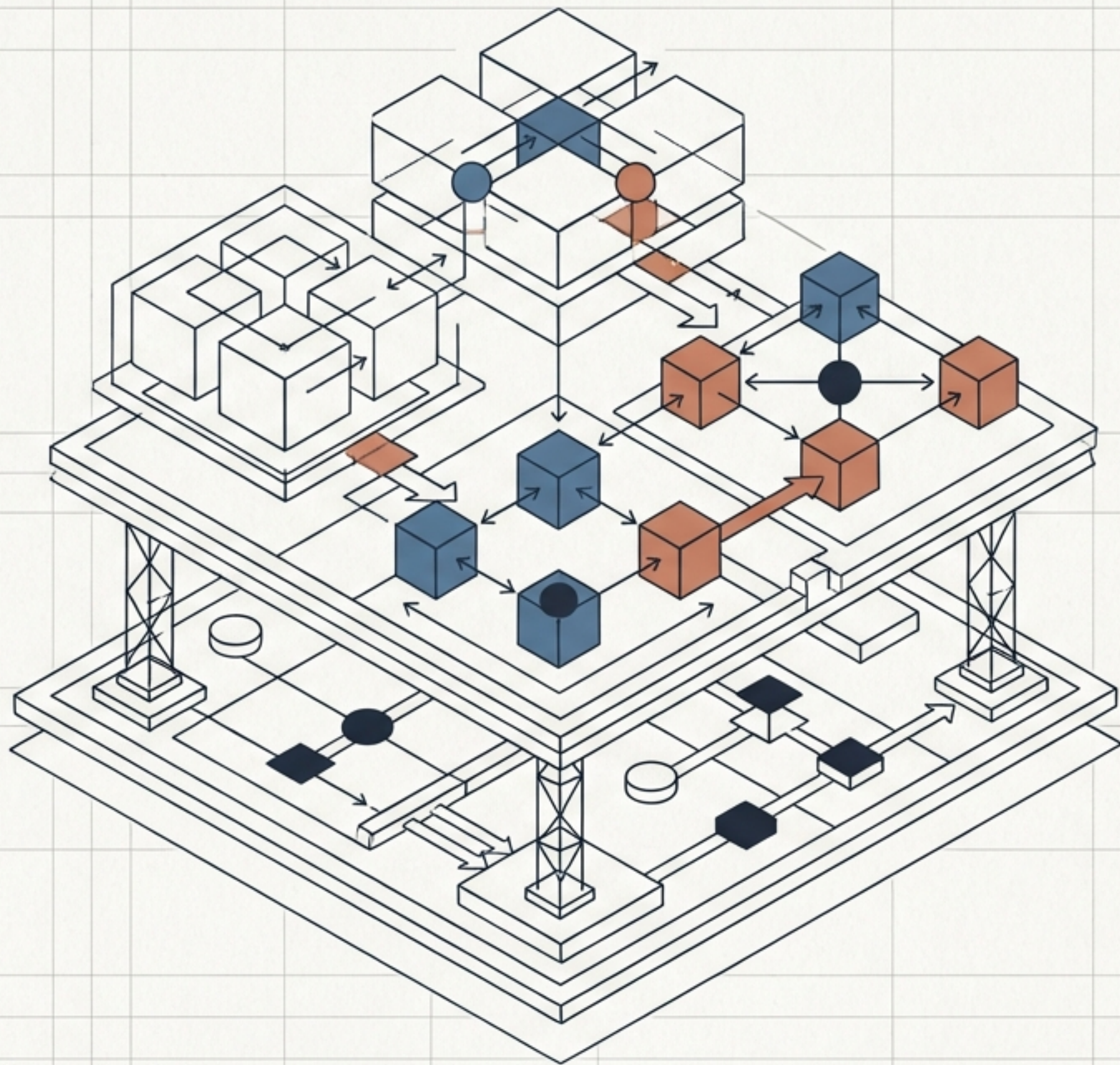


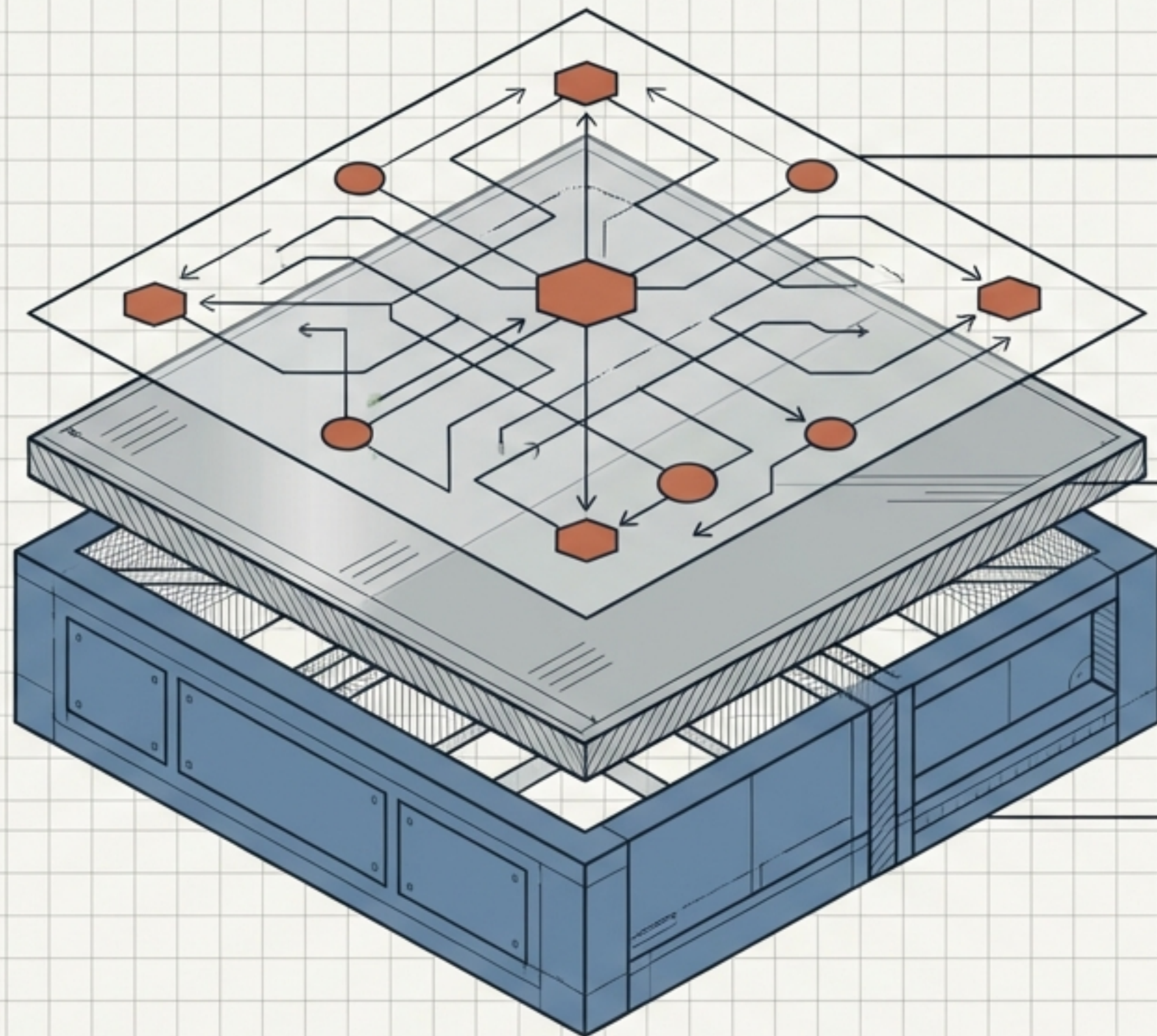
웹 기술의 진화와 2026 아키텍처 비전

단순 구현(Implementation)에서
시스템 설계(Architecture Design)로의
패러다임 전환

핵심 웹 파운데이션부터 AI 네이티브 및
서버 중심(Server-First) 미래 트렌드 종합 브리핑



웹 아키텍처의 3대 근간 (The Web Trinity)



JavaScript (동적 제어):

JIT 컴파일 기반의 로직.
DOM 조작, 네트워크 통신(AJAX),
사용자 인터랙션의 핵심.

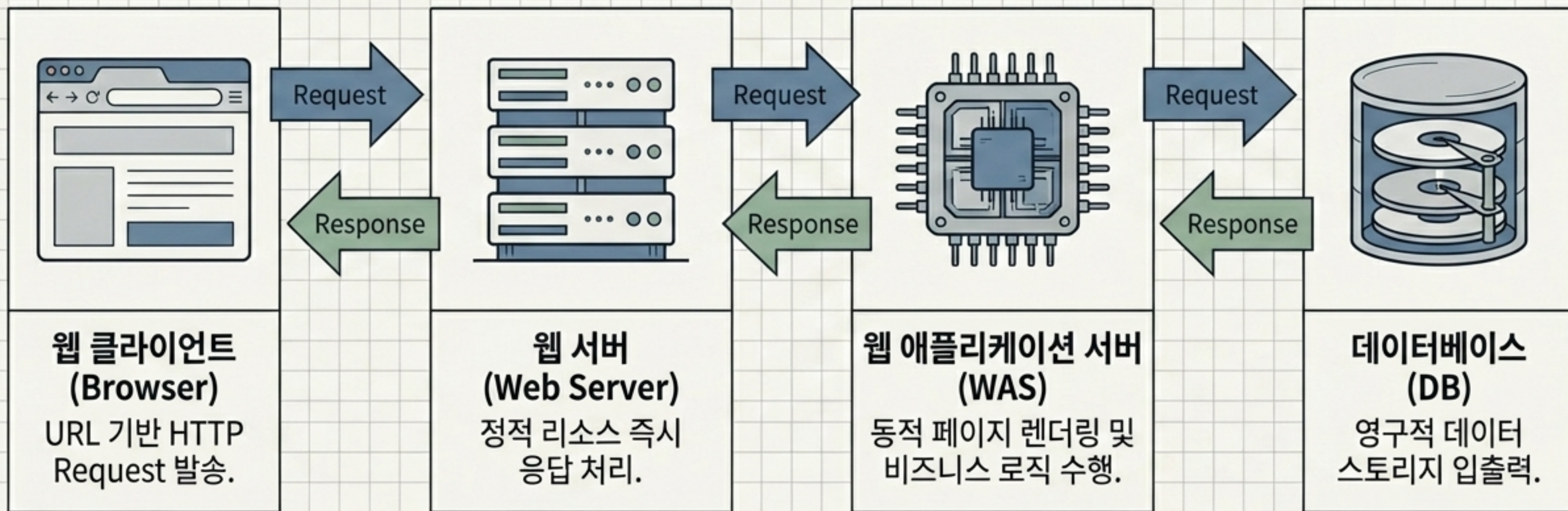
CSS (시각적 표현):

선택자(Selector) 기반의 스타일링.
구조와 디자인의 완벽한 분리.

HTML (의미와 구조):

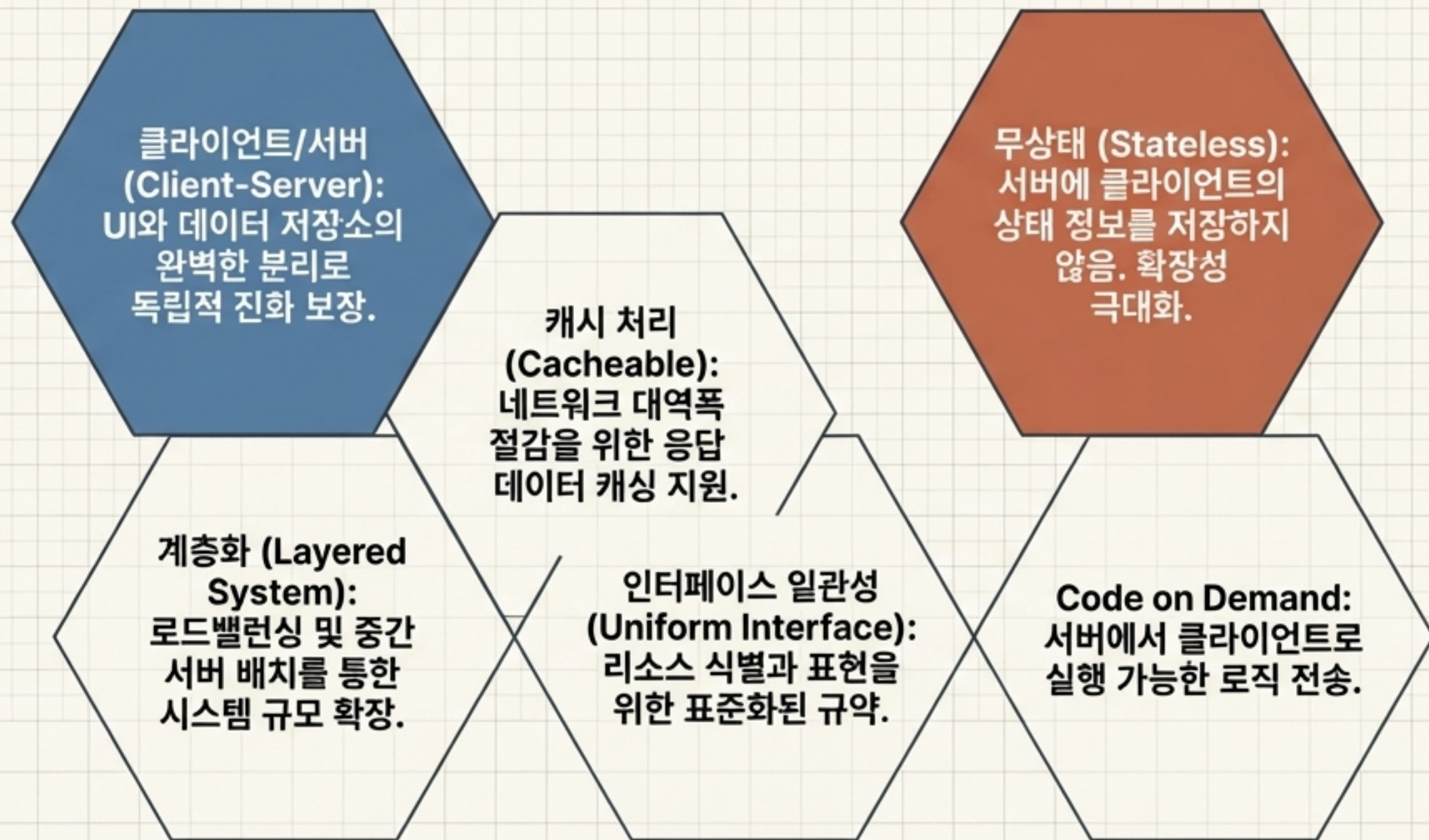
문서의 마크업 베이스라인.
요소(Element)와 속성(Attribute)을
통한 정보의 계층적 정의.

웹 서비스의 동적 데이터 파이프라인



분산 시스템의 표준 규약: REST 아키텍처

1989년 팀 버너스리의 웹 발명 이후 고도화된 하이퍼미디어 시스템 제약 조건 6가지



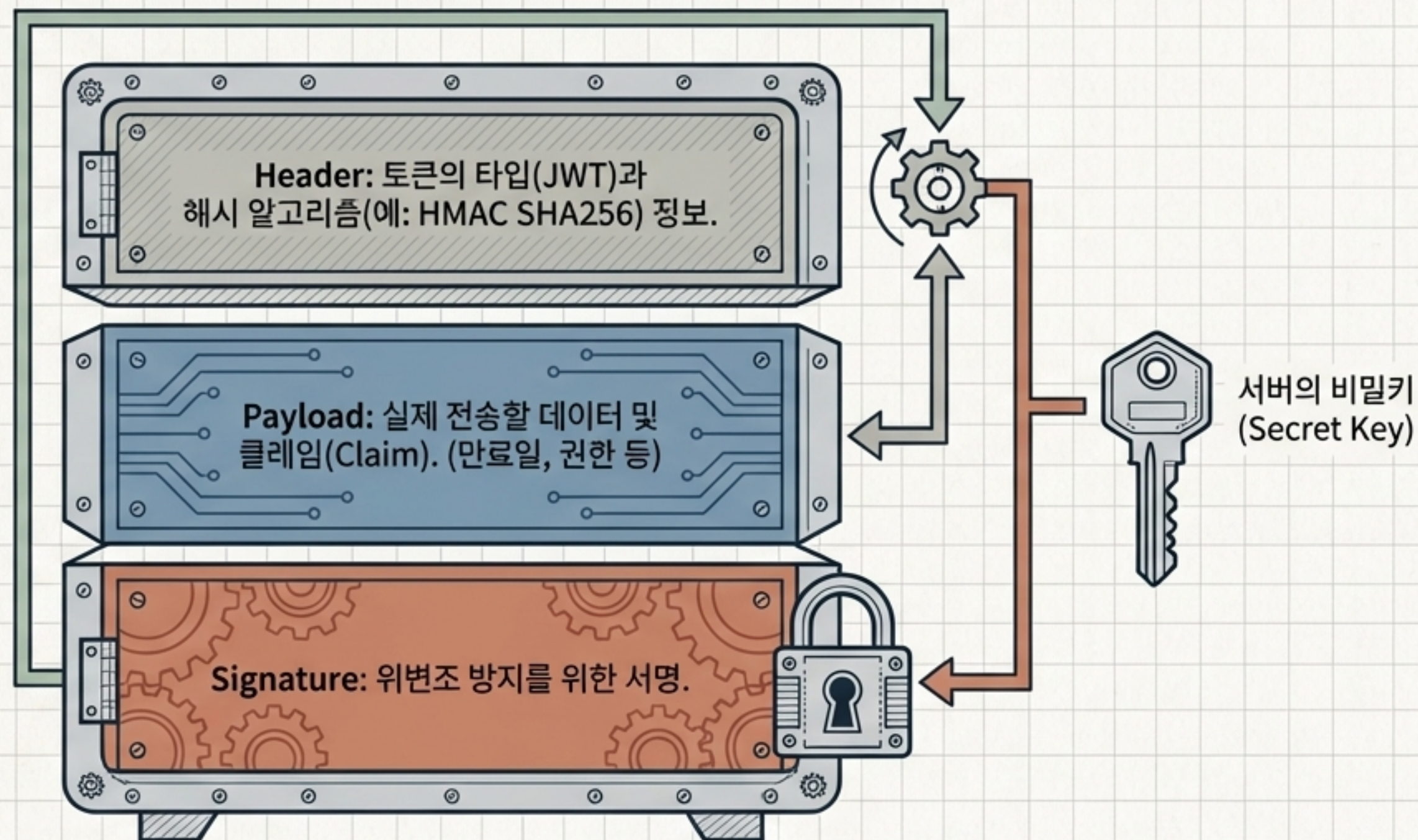
상태 관리의 진화: 인증 아키텍처 비교 분석

	쿠키 (Cookie)	세션 (Session)	토큰 (Token/JWT)
저장 위치	클라이언트(브라우저)	서버	클라이언트 (별도 저장소 불필요)
핵심 특징	Key-Value 문자열	서버 측 민감 정보 중앙 관리	서버의 무상태성(Stateless) 완벽 유지
취약점/단점	보안 취약 및 용량 제한	요청 증가 시 서버 메모리 부하 급증	탈취 시 즉각적인 무효화(Revoke) 어려움

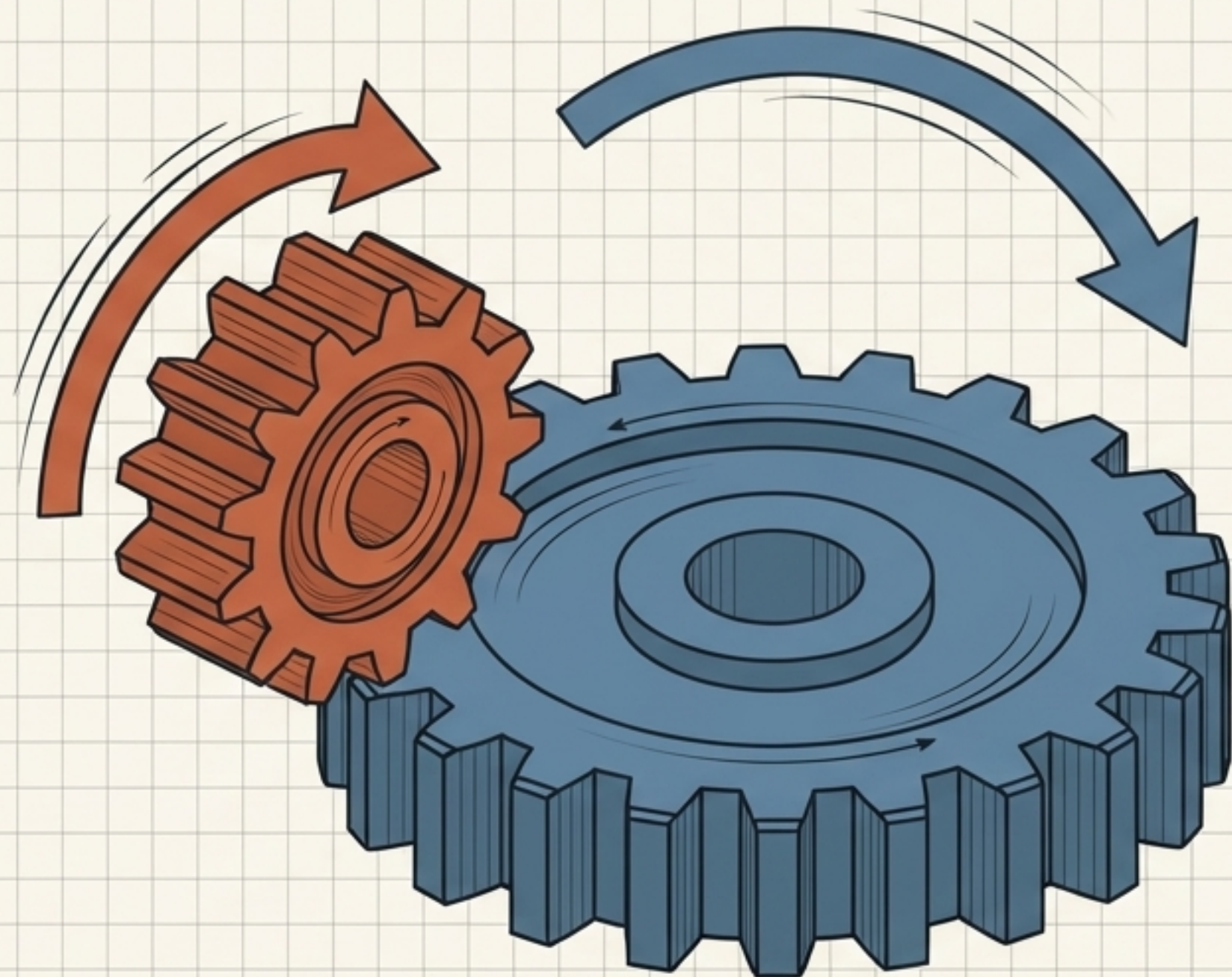
현대 웹은 서버 스케일아웃을 위해 서버 부하를 제거하는
'무상태(Stateless)' 기반의 Token 아키텍처를 표준으로 채택함.

JWT (JSON Web Token)의 구조적 암호화 원리

데이터 자체를 숨기는 것이 아니라, '신뢰성(조작되지 않음)'을 증명하는 무결성 검증 아키텍처.



보안과 사용성의 결합: 이중 토큰 체계



Access Token (작은 톱니바퀴)

특징: 매우 짧은 수명(Lifespan).
높은 빈도로 API 요청에 사용.

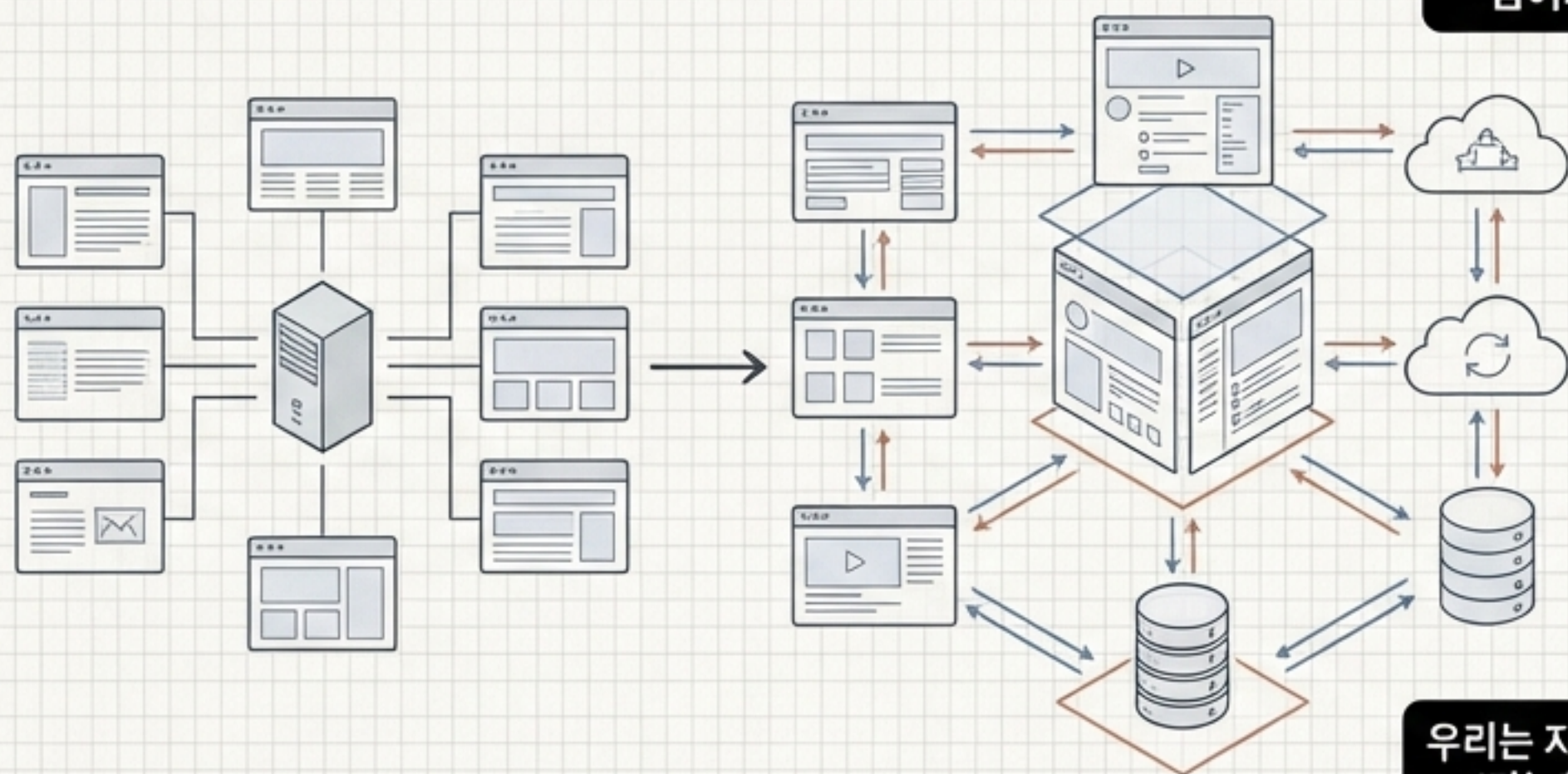
목적: 탈취되더라도 짧은 시간 안에 만료되어
피해 최소화.

Refresh Token (큰 톱니바퀴)

특징: 긴 수명. 안전한 스토리지에 보관.

목적: 오직 작은 톱니바퀴(Access Token)가
멈췄을 때(만료 시) 이를 다시
회전(재발급)시키기 위한 용도로만 사용.

패러다임의 진화: Web 1.0에서 지능형 유비쿼터스로

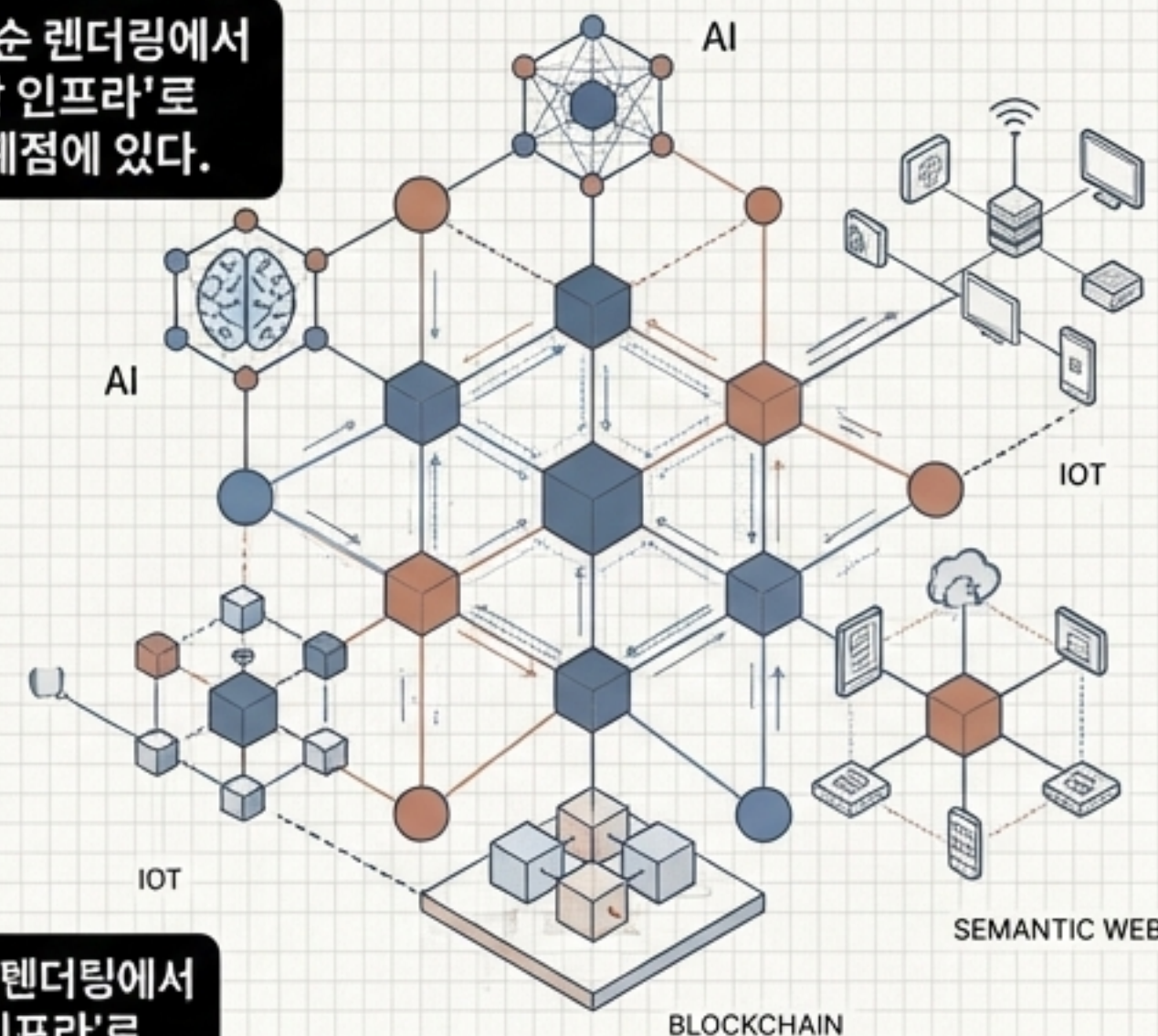


Web 1.0 (1989~2004)

정적 페이지 중심의 '읽기' 전용 웹.
소수 생산자, 다수 소비자.

Web 2.0 (2004~현재)

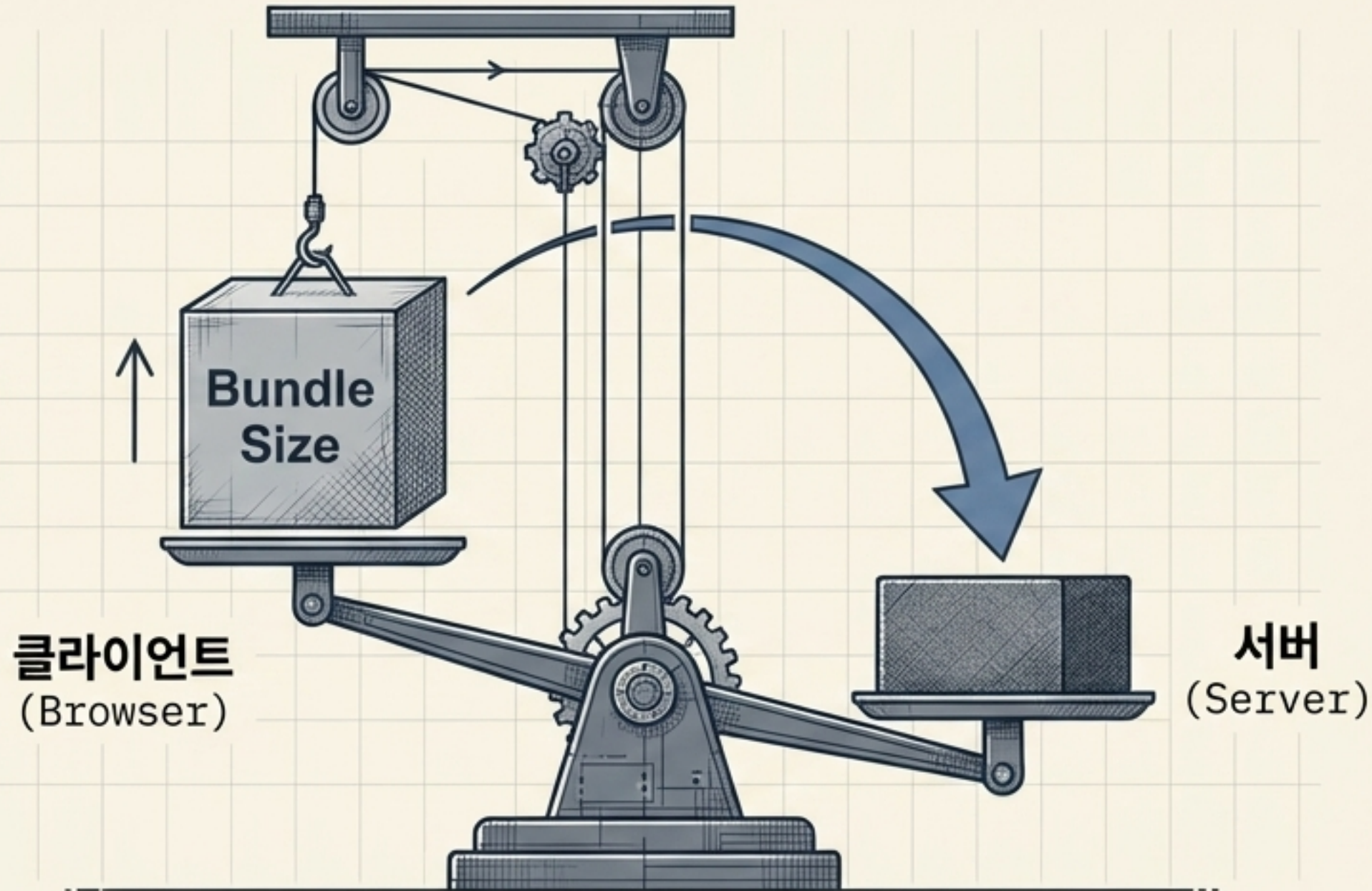
참여형 웹(UGC). 블로그, SNS.
Ajax 기술을 통한 동적 인터랙션의 폭발적 증가.



Web 3.0 & 4.0 (미래)

AI 에이전트, 시맨틱 웹, Web3(블록체인), IoT가
결합된 지능형 유비쿼터스 컴퓨팅 환경.

무게 중심의 이동: 서버 중심(Server-First) 패러다임



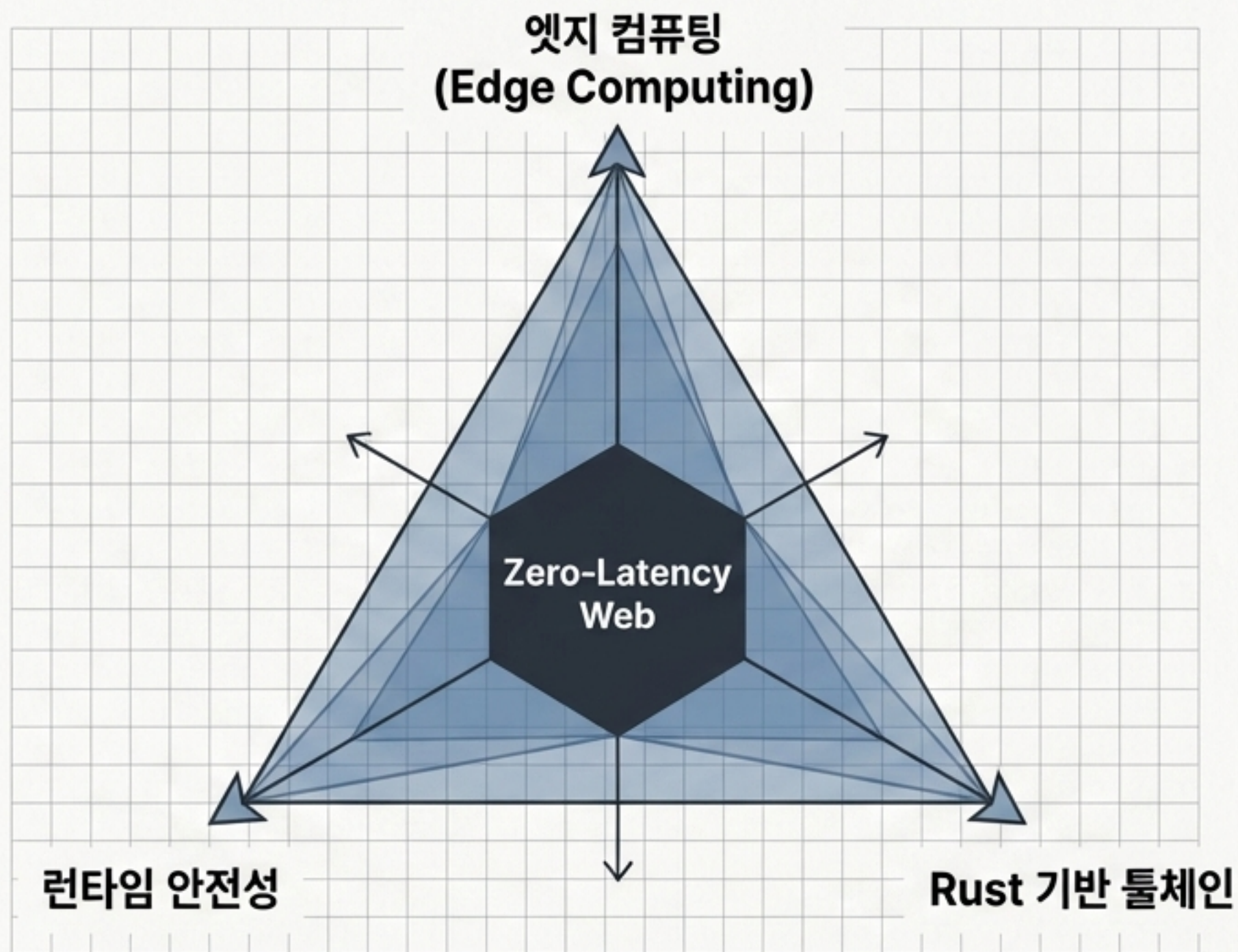
React 19 & Next.js 15/16:
클라이언트에서 서버로 개발 중심축 이동.

React Server Components (RSC): 백엔드 로직을 컴포넌트 내부로 직접 통합.

풀스택 생산성: Next.js의 서버 액션과 캐시 API를 통해 프론트엔드와 백엔드가 단일 코드베이스로 결합.

클라이언트 측 번들(Bundle) 사이즈 최대 45% 감소

차세대 인프라: 한계 없는 성능과 안정성



엣지 컴퓨팅 (Edge Computing)

Cloudflare, Vercel 미들웨어 도입.
사용자와 가장 가까운 노드에서 요청을 선제적으로
처리하여 레이턴시를 50ms 미만으로 극적 단축.

Rust 기반 튜체인

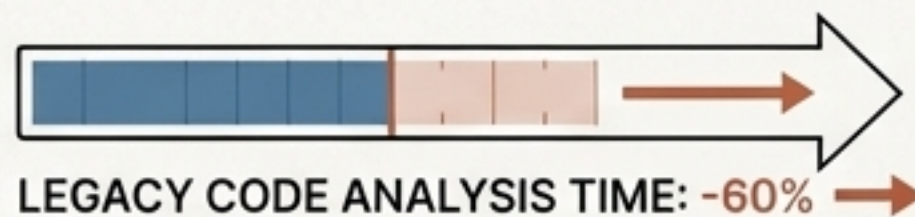
Turbopack, Rspack 등 차세대 컴파일러 도입.
기존 JavaScript 도구 대비 압도적인 빌드 속도로
개발 생산성 혁신.

런타임 안전성

TypeScript의 정적 타입을 넘어 Zod, Valibot을
활용한 API 응답 데이터 실시간 검증 필수화.

구현의 자동화: AI 네이티브 개발 환경

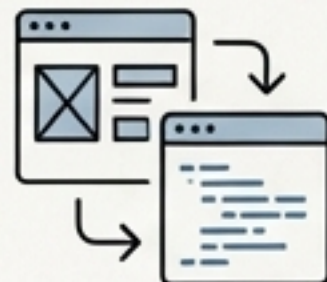
AI Native IDE



Cursor, Windsurf 도입.
전체 프로젝트 컨텍스트를 이해하는 에이전트.

레거시 코드 분석 시간 60% 이상 단축

Design-to-Code



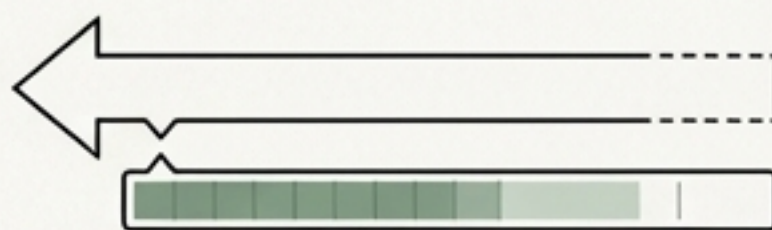
Figma Dev Mode, v0.
UI 설계에서 코드로의 다이렉트 변환.

퍼블리싱 업무의 80% 자동화

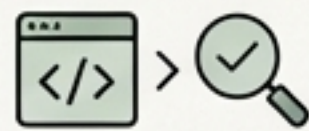
Shift Left Testing



BLOCK



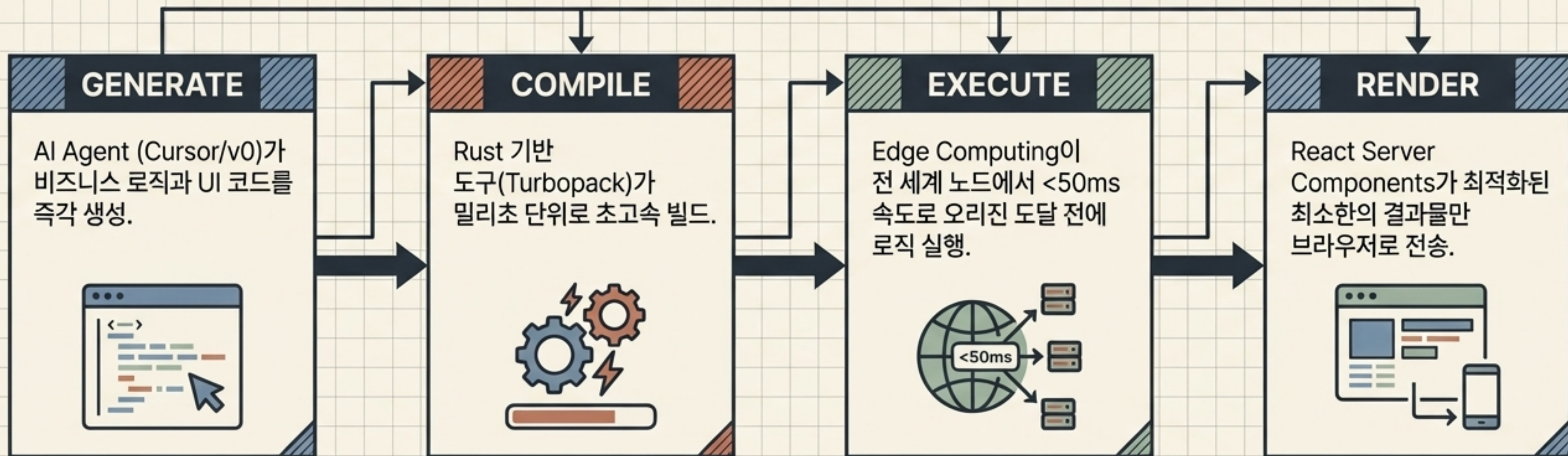
DEVELOPMENT



TESTS

AI 에이전트를 통한 코드 리뷰 및 유닛 테스트 자동화.
개발 초기 단계에서 보안 취약점과 버그를 원천 차단.

The 2026 Web Ecosystem: 초연결 렌더링 파이프라인



개별 기술의 발전이 아닌, '구현 시간 제로'와
'레이턴시 제로'를 향한 하나의 거대한 통합 시스템.

결론: 구현자(Implementer)에서 설계자(Architect)로

과거의 룰: Implementation



■
단순 코드 작성, 반복적인 UI 퍼블리싱, 개별 버그 픽스.
(이제 이 영역은 AI의 몫이 됨)
■

2026년의 핵심 역량: Architecture Design



-
- 시스템 조망: 프론트/백엔드/엣지를 아우르는 전체 아키텍처 설계.
 - AI 검증 및 오케스트레이션: AI가 생성한 결과물을 비판적으로 분석하고 검증.
 - 비즈니스 로직 고도화: 기술 변화 속도에 매몰되지 않고 조직에 최적화된 솔루션 도입.
-

미래의 개발자는 코드를 타이핑하는 사람이 아니라, AI라는 거대한 엔진을 설계하고 조향하는 아키텍트다.